

UNIVERSITY OF BIRMINGHAM

School of Computer Science

Security of Real-World Systems

Main Summer Examinations 2022

Security of Real-World Systems

Question 1

- (a) The EMV protocol involves the reader sending an unpredictable, random number (UN) to the card, which is then used to generate a proof of payment AC. Briefly describe what attacks would be possible if this was instead a fixed constant value.

[4 marks]

- (b) A passive eavesdropper can see all of the data being transmitted and so learn the identity of the card (the PAN) and the amount being paid. Design a simple extension to this protocol that protects the payer's privacy from passive eavesdroppers. Write your protocol in the Alice and Bob notation used in the lectures. You may leave out steps that have no effect on the security of the protocol (such as the select messages). Explain why your extension stops a passive eavesdropper from learning any useful data.

[6 marks]

- (c) The WireGuard protocol provides post compromise security by regularly "rekeying", this means that an attacker that has learnt the secret state of the protocol must continue to actively man-in-the-middle the protocol to be able to continue to decrypt the messages.

The WireGuard protocol provides no protection against an attacker that learns the secret state of the protocol and also controls the communication link between the two parties using the protocol. However, if the parties running the protocol had access to a second "out of band" communication link, not accessible by the attacker (e.g. direct NFC communication between two phones) it would be possible to stop such attacks.

Describe how such an out of band channel could be used to detect if an active attacker has compromised the protocol. Be clear on exactly what should be sent across the out of band channel and how this stops the attack.

[10 marks]

Question 2

A smartcard uses a challenge-response protocol to grant access only to readers that can prove that they hold a certain private 2048-bit () RSA key sk . The smartcard only stores the corresponding RSA public key pk . The protocol works as follows:

- The smartcard sends a random 32-byte nonce x to the reader.
- The reader signs the nonce with the private key, i.e., computes $y = \text{sign}_{sk}(x)$.
- The reader sends the signature y to the smartcard. The smartcard verifies the signature y , and if it is valid, allows the reader to access further functionality.

The C (pseudo)code of the function implementing this on the smartcard is as follows:

```
1
2 // Functions random() and receive() left out here
3
4 void send(char* b, size_t len) {
5     size_t i = 0;
6     while(i != len) {
7         send_byte(b[i]); // send a single byte
8         i++;
9     }
10 }
11
12 void smartcard_main() {
13     char secret_data[16] = { ... };
14     char nonce[32];
15     char signature[256];
16     int valid = 1;
17
18     random(nonce, 32); // generate 32 random bytes in nonce
19     send(nonce, 32);   // send nonce to reader
20     receive(signature, 256); // get max. 256 byte from reader
21     // This function returns 1 if the signature on the nonce
22     // is valid given the public key pk, otherwise 0
23     valid = rsa_verify(signature, nonce, pk);
24
25     if(!valid)
26         return;
27
28     // Else: continue and give access to secret_data
29     send(secret_data, 16);
30 }
```

You find that by injecting a voltage glitch, you can fault specific instructions on the smartcard. More exactly, the fault can cause either of the following three effects:

- **SKIP**: Skip return instructions and function calls;
 - **FIX**: Set the least-significant byte of a variable to 0xF0;
 - **COMPARE**: Make a comparison (e.g., $a < b$ or $a \neq b$) return the opposite result (e.g., $1 < 2$ would evaluate to false).
- (a) Assume that the RSA implementation uses the Chinese Remainder Theorem (CRT) optimization. Someone claims that a fault attack on the smartcard can thus reveal the private key. Is this true? Briefly explain your answer. **[3 marks]**
- (b) Using the fault model from above, show *two* ways for how an attacker can get access to `secret_data` with a single fault. Assume that data on the stack is stored at decreasing addresses, i.e., `nonce` is at a lower address than `secret_data`. Further assume that the functions that are not specified (`random()`, `receive()`, `send_byte()`, `rsa_verify()`) are immune to fault injection. For both of your attacks, indicate with line in the above code is affected by the fault, which fault type (SKIP, FIX, COMPARE) you assume, and briefly explain why the respective attack works. **[8 marks]**
- (c) For each of your attacks from (b), modify the code such that an attack with a *single fault* is no longer possible. Only give the part of the code (usually one line) that you changed. Briefly explain why your countermeasure works. **[6 marks]**
- (d) Assume now that fault injection can only be applied to the RSA implementation but not the surrounding checks. Also assume that the fault allows you to set the 1700 most-significant bits of the RSA modulus n to zero, resulting in a modulus n' . Also assume that the public exponent e has an inverse mod $\phi(n')$. Explain how this scenario can be used to forge a valid signature. **[3 marks]**

Question 3

You analyse an implementation of a block cipher. To save memory space, the programmer decided to compute the inverse S-Box for decryption on-the-fly as follows:

```
1 // sbox[] is a 256-byte array holding the forward S-Box
2 char inv_S(char val) {
3     for(int i = 0; i < 256; i++) {
4         if(val == sbox[i])
5             return i;
6     }
7     return 0;
8 }
```

In the first round of the decryption implementation, the inverse S-Box is then applied to each byte c_i ($0 \leq i \leq 7$) of the ciphertext individually after the respective key byte k_i has been XORed, yielding the intermediate value z_i :

$$z_i = \text{inv_S}(c_i \oplus k_i)$$

- (a) Assume an attacker can get the runtime of the single of `inv_S()` for the first ciphertext byte c_0 only (e.g. from a power trace). In other words, the attacker learns how many cycles the execution of `inv_S( $c_0 \oplus k_0$ )` took. Explain how the attacker can then recover the first key byte k_0 . How many iterations (different values for c_0) are needed? **[8 marks]**
- (b) You fix this issue by not computing the inverse S-Box on the fly, but instead pre-computing another lookup table `inv_sbox[]` that is then used to compute $z_i = \text{inv_sbox}[c_i \oplus k_i]$. Now assume that the attacker runs in another (unprivileged) process on the same machine as a program using the decryption and that the decryption library (including the inverse S-Box) is in a shared library. Explain how an attacker that interacts with the decryption process can leak the first key byte k_0 through a flush-and-reload cache attack. Each cache line is 64 bytes long, so `inv_sbox[]` spans four cache lines. How many iterations (different values for c_0) are needed on average? **[8 marks]**
- (c) To fix this issue, you successfully replaced the table lookup for the S-Box with a series of Boolean operations (NOT, AND, OR, XOR) that are all executed in constant time. Does this prevent (i) timing attacks, (ii) cache attacks, and (iii) differential power analysis (assuming an attacker who can measure the power consumption)? In each case, explain your answer in one sentence. **[4 marks]**